SAMK: Secure Aggregation for Federated Learning Under Multiple Keys With Low Communication Rounds

Cheng Guo, *Member, IEEE*, Ling Li, Ximeng Liu, *Senior Member, IEEE*, Kim-Kwang Raymond Choo, *Senior Member, IEEE*, and Yining Liu

***Abstract*—While multi-key homomorphic encryption (MKHE) ensures privacy in federated learning (FL) by encrypting model updates, its requirement for aggregate ciphertext decryption and dropout handling increases communication rounds during aggregation. To enable secure multi-key aggregation with low communication rounds, we propose SAMK. SAMK enables the server to compute the sum of model updates from clients participating in FL, while keeping these updates encrypted by different keys throughout the computation. By utilizing the polynomial property of the BFV ciphertext, SAMK successfully implements individual decryption of the aggregated ciphertext (encrypted under multiple keys) by each client using their respective keys, resulting in low communication rounds. It means that in SAMK, all client interactions are avoided and the client-server interaction is only once (ciphertext uploads and downloads) in each round of aggregation computation. In addition, SAMK is robust to any number of clients dropping out at any time, and the client who has dropped out after uploading model updates, can still get the correct aggregation result upon reconnecting. We prove the security of SAMK for semi-honest server and clients, where client collusion is also considered. At last, we implement SAMK and comprehensively evaluate its performance to demonstrate its practicability.**

***Index Terms*—Secure aggregation, multiple keys, federated learning (FL), homomorphic encryption, privacy preserving.**

Received 17 January 2024; revised 19 June 2025; accepted 20 August 2025. Date of publication 26 August 2025; date of current version 4 November 2025. This article was supported in part by the National Science Foundation of China under Grant 61871064, Grant 61501080, and Grant 62071320, in part by the Guangxi Key Laboratory of Trusted Software under Grant KX202026 and in part by CAAI-Huawei MindSpore Open Fund. The work of Kim-Kwang Raymond Choo was supported by the Cloud Technology Endowed Professorship. *(Corresponding author: Kim-Kwang Raymond Choo.)*

Cheng Guo is with the School of Software Technology, Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, Dalian University of Technology, Dalian 116620, China (e-mail: guocheng@dlut.edu.cn).

Ling Li is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, School of Software Technology, Dalian University of Technology, Dalian 116620, China (e-mail: liling_li_ling@163.com).

Ximeng Liu is with the College of Mathematics and Computer Science/College of Software, Fuzhou University, Fuzhou 350108, China (e-mail: snbnix@gmail.com).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cybersecurity, University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: raymond.choo@fulbrightmail.org).

Yining Liu is with the Guangxi Key Laboratory of Trusted Software, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: ynliu@guet.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3602908

I. Introduction

FEDERATED learning, an emerging machine learning paradigm, has garnered significant attention due to its capability to preserve data privacy. Unlike traditional machine learning, which requires clients to upload training data to the server for centralized training, FL enables clients to train locally and upload only their local model updates to the server for aggregation to obtain a global model. Data privacy is considered to be preserved by keeping training data locally. However, recent studies have revealed that data privacy can still be compromised due to various privacy attacks, such as inversion attacks and membership inference attacks [1]. These attacks can reconstruct training data or obtain relevant privacy information by analyzing model updates [2], [3]. Therefore, protecting the privacy of the model updates is necessary to achieve robust data privacy.

In order to protect the model updates without impacting the aggregation computation, secure aggregation for FL was introduced in [4] and has been further developed in recent years. Secure aggregation for FL refers to the problem of computing the sum of model updates without revealing them to any parties, i.e., clients or server. Currently, secure aggregation for FL is mainly based on differential privacy (DP) [5], secure multi-party computation (MPC) [4], and homomorphic encryption (HE) [6]. Secure aggregation based on DP preserves the privacy of model updates by adding noise to them, which is efficient, but it may introduce errors that compromise the accuracy of the aggregation results. In contrast, secure aggregation based on MPC and HE can achieve higher accuracy in aggregation results. Secure aggregation based on MPC uses various MPC techniques (e.g., garbled circuits [7], secret sharing [8] and double-masking [4], [9], [10]) to compute the sum of model updates without revealing them. However, it generally requires many interactions between entities [11], and clients may have to be online continuously to interact. Secure aggregation based on HE can avoid this problem.

Secure aggregation based on HE uses homomorphic cryptographic algorithms to protect model updates and performs homomorphic operations during aggregation. The HE-based secure aggregation in FL can be divided into three steps: 1) clients encrypt model updates themselves using homomorphic cryptographic algorithm and send homomorphic ciphertexts to the server, 2) the server performs additive homomorphic computation on the ciphertexts to obtain the aggregated ciphertext, and

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

sends it to clients, 3) clients decrypt the aggregated ciphertext themselves to obtain the aggregated result. These three steps require only one interaction per client and server, and clients can go offline anytime. However, these steps only apply to the case where all clients use the same encryption key. In this case, insider attacks may occur, where semi-honest clients can obtain other clients' model updates by decrypting their ciphertexts using the shared key. Therefore, all clients need to use different keys.

When clients use different keys, existing schemes add additional steps beyond the above three-step process to accomplish secure aggregation, resulting in an increase in communication rounds. This increase primarily stems from the additional inter-client communication required to address the following two critical requirements: **1) Aggregated ciphertext decryption**, there are two reasons for increased inter-entity communication and they both aim to enable clients to decrypt the aggregated ciphertext. a) Aggregating ciphertexts encrypted under distinct client keys produces a result decryptable only with all corresponding secret keys. As clients retain their keys privately, collaborative decryption via communication is required. b) Some schemes securely re-encrypt ciphertexts during aggregation through multi-party collaboration. This transformation allows each client to decrypt the corresponding aggregated ciphertext but inherently necessitates additional communication rounds between the server and clients/auxiliary servers. **2) Dropout handling**: In practice, clients are not always online. When some clients drop out during aggregation, the absence of the dropouts' keys can prevent the aggregation computation from completing correctly or the aggregation result from being obtained correctly. Existing schemes address this through additional inter-entity interaction to collaboratively eliminate the impact of the dropouts' keys, ensuring that the aggregation is completed securely and correctly.

To achieve secure multi-key aggregation without additional communication rounds, we propose SAMK. This scheme implements the above three-step process, enabling secure aggregation computation under multiple keys while maintaining the original communication rounds. More precisely, we have considered the following factors and challenges in SAMK:

1) *Aggregate ciphertext independent decryption:* The multi-key aggregated ciphertext contains the keys of all clients participating in FL. To eliminate the communication rounds for collaborative decryption, an independent decryption mechanism is essential. This mechanism must enable any client to decrypt the aggregated ciphertext using only its own key without the participation of other clients. Since each client only knows its own key, it is challenging for them to decrypt the aggregated ciphertext (encrypted under multiple keys) individually.

2) *Secure aggregation with limited interaction:* To minimize communication rounds in secure aggregation computation, the computation process is strictly limited to the above three-step process, i.e., the client encrypts and decrypts individually, and the server is only one and aggregates individually. This process results in low communication rounds, i.e., there is no interaction between the clients and only one interaction (ciphertext uploads and downloads) between each client and the server.

3) *Dropout Handling without interaction:* To eliminate the communication rounds for dropout handling, a non-interactive dropout handling mechanism needs to be designed. In the case of any client dropout, this mechanism must enable the server to aggregate the ciphertexts independently and correctly, and the online clients to decrypt the aggregated results independently and correctly, without inter-entity collaboration or leakage of dropout clients' private information.

Existing research has explored secure aggregation under multiple keys, but to the best of our knowledge, none have addressed the mixture of constraints presented above.

A. Related Works

Since Bonawitz et al. [4] presented the problem of secure aggregation for FL and proposed secure aggregation based on double masking, which belongs to MPC technology, numerous new schemes have increasingly emerged. Secure aggregation based on HE has garnered significant interest due to its high accuracy of aggregated results and low interaction requirements.

Existing secure aggregation schemes based on HE are mainly rely on paillier cryptosystem [12], BFV cryptosystem [13], and CKKS cryptosystem [14]. Some schemes were primarily designed for single-key scenarios [6], [15], [16], [17], [18]. Scheme [6] encrypts the model updates using additive homomorphic cryptography and instantiates it with Learning with Errors (LWE)-based cryptosystem and paillier. Both [15] and [16] use paillier to encrypt model updates, while [17] and [18] use CKKS for this purpose. As they failed to defend against insider attacks, more and more works focused on multi-key scenarios.

For multi-key scenarios, secure aggregation schemes based on HE currently has 4 main types: works based on sharing public key [19], [20], works based on two non-colluding servers [21], [22], [23], [24], [25], [26], works based on two-trapdoor cryptosystem [27], [28], works based on multi-key homomorphic encryption (MKHE) [29], [30].

Wang et al. [19] and Ma et al. [20] used a shared public key for encryption, while decryption requires the collaboration of all clients. In [19], the shared public key is distributed by a trusted authority, and its corresponding secret key is split into copies sent to clients separately as their own secret key. In [20], the shared public key is generated by aggregating all clients' public keys, which the clients perform without needing a trusted organization. The above schemes allow the server to learn the aggregated result and cannot resist semi-honest server attacks. Zhang et al. [21] and Jiang et al. [22] used the double decryption mechanism of BCP [31], allowing two servers to collaborate on aggregation while not getting the aggregation results. Scheme [23] is the same idea as these two schemes, except that it used linearly homomorphic encryption (LHE) instead of BCP cryptosystem. Schemes [24], [25], [26] also used two servers. In [24], one server aggregates the ciphertext and the other collects auxiliary information for decryption, while in [25] and [26], one server aggregates the ciphertext and re-encrypts it, and the other encrypts it again. These schemes require that the two servers cannot collude with each other, which is difficult to achieve in reality. The schemes proposed by Ma et al. [27] and Zhang et al. [28] do not need to consider server collusion. They implement secure aggregation through collaboration between clients and a server without additional servers. However, these

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

schemes increase the interaction between client and server. Cai et al. [29] designed the trusted execution environment (TEE)-based multi-key HE cryptosystem for secure aggregation, but constructing TEE often brings additional overhead. Hao et al. [30] used a multi-key variant of CKKS [32] to achieve secure aggregation by allowing different clients to encrypt model update with different keys. However, this scheme requires collaborative decryption among clients, thus increasing the interaction between clients.

B. Our Contributions

We design SAMK, a secure aggregation scheme for FL under multiple keys without frequent interactions. Our main contributions include:

1) SAMK is constructed based on BFV and paillier HE. It allows different clients to encrypt their respective model updates using their own BFV keys and decrypt the aggregated ciphertext (containing the keys of all participating clients) individually using their keys. Throughout the aggregation, the client's model update ciphertext is never decrypted.

2) Compared to existing secure aggregation based on HE under multiple keys, there is no interaction between clients and only 1 interaction between each client and the server in SAMK. In addition, each client can be offline at any time without affecting any operation of other entities, and all online clients can obtain the same correct aggregation result. If the client is dropped after uploading the ciphertext, it will still get the correct aggregation result after going online.

3) We have demonstrated that SAMK can resist attacks from both honest-but-curious clients and the server. Additionally, SAMK can tolerate collusion of up to $n-2$ clients when n clients have uploaded ciphertexts. This implies that even if $n-2$ clients collude and possess knowledge, including each other's model updates and aggregation results, they cannot learn the model updates of the remaining 2 clients.

C. Organization

Section II gives some preliminaries. Section III presents an overview of SAMK design, followed by the detailed construction of SAMK in Section IV. Then, the security and performance analysis of SAMK are discussed in Sections V and VI, respectively. Finally, we conclude this work in Section VII.

II. Preliminaries

A. BFV Cryptosystem

The Brakerski-Fan-Vercauteren (BFV) cryptosystem [13] is a ring learning with errors (RLWE) scheme that supports both additive and multiplicative homomorphic operations. The ciphertext space is the polynomial quotient ring $\mathcal{R}_q = \mathbb{Z}_q[x]/X^N+1$, where N is a power of 2 and q is ciphertext modulus. The plaintext space is the polynomial quotient ring $\mathcal{R}_t = \mathbb{Z}_t[x]/X^N+1$, where t is plaintext modulus and $t<q$. The RLWE error distribution χ over $\mathcal{R}_q$ has coefficients distributed according to a centred discrete gaussian with standard deviation σ and truncated support over $[-B, B]$. The key distribution φ over $\mathcal{R}_3$ has coefficients uniformly distributed in $\{-1, 0, 1\}$. We focus on the following algorithm:

• BFV . $\mathrm{Setup}(1^\lambda) \rightarrow pp$: For a given security parameter λ, set the RLWE dimension N, ciphertext modulus q, plaintext modulus t, key distribution φ and the RLWE error distribution χ. Output the public parameter pp:

$$pp = \{N, q, t, \varphi, \chi\}$$

• BFV . $\mathrm{SecKeyGen}(pp) \rightarrow sk$: Given the security parameter λ, sample $s \leftarrow \mathcal{R}_3$. Output secret key sk:

$$sk = s$$

• BFV . $\mathrm{PubKeyGen}(pp, sk) \rightarrow pk$: Sample $a \in \mathcal{R}_q$ and $e \leftarrow \chi$. Output public key pk:

$$pk = (p_0, p_1) = ([-(s \cdot a + e)]_q, a)$$

• BFV . $\mathrm{Encrypt}(pp, pk, m) \rightarrow ct$: Given plaintext m, sample $u \leftarrow \mathcal{R}_3$ and $e_0, e_1 \leftarrow \chi$. Computer $\Delta = \lfloor q/t \rfloor$. Output ciphertext ct:

$$ct = (c_0, c_1) = ([\Delta[m]_t + up_0 + e_0]_q, [up_1 + e_1]_q) \in \mathcal{R}_q^2$$

• BFV . $\mathrm{Decrypt}(pp, sk, ct) \rightarrow m$: Output plaintext m:

$$m = \left[\left\lfloor \frac{t \cdot [c_0 + c_1 s]_q}{q} \right\rceil \right]_t$$

The multi-key BFV (MK-BFV) cryptosystem [32] modifies standard BFV to support ciphertexts encrypted under different keys while enabling homomorphic addition and multiplication. Assuming that there are k participants, each with an index $i (1 \leq i \leq k)$ to its public and secret keys. We mainly use the following algorithms:

• MK-BFV . $\mathrm{Setup}(1^\lambda) \rightarrow pp$: Run BFV . $\mathrm{Setup}(1^\lambda)$ and output pp.

• MK-BFV. $\mathrm{KeyGen}(pp) \rightarrow \{sk_i, pk_i\}$: Each participant i generates secret and public keys by $sk_i \leftarrow$ BFV . $\mathrm{SecKeyGen}(pp)$ and $pk_i \leftarrow$ BFV . $\mathrm{PubKeyGen}(pp, sk_i)$, respectively.

• MK-BFV . $\mathrm{Encrypt}(pp, pk_i, m_i) \rightarrow \overline{ct}_i$: Each participant i generates BFV ciphertext by $ct_i = (c_{i,0}, c_{i,1}) \leftarrow$ BFV . $\mathrm{Encrypt}(pp, pk_i, m_i)$. Then ct_i is converted into the ciphertext $\overline{ct}_i$:

$$\overline{ct}_i = (c_{i,0}, \bar{c}_{i,1}, \ldots, \bar{c}_{i,k}) \in \mathcal{R}_q^{k+1}$$

where $\bar{c}_{i,j} = \begin{cases} c_{i,1} & \text{if } j = i, \\ 0 & \text{otherwise.} \end{cases}$ for $1 \leq j \leq k$.

• MK-BFV . $\mathrm{Decrypt}(pp, \overline{ct}_i, sk_1, \ldots, sk_k) \rightarrow m_i$: Output plaintext m_i:

$$m_i = \left[\left\lfloor \frac{t \cdot \left[c_{i,0} + \sum_{j=1}^{k} \bar{c}_{i,j} \cdot sk_j \right]_q}{q} \right\rceil \right]_t$$

• MK-BFV . $\mathrm{Add}(pp, \overline{ct}_0, \overline{ct}_1) \rightarrow ct_{add}$: Given two ciphertexts: $\overline{ct}_1 = (c_{1,0}, c_{1,1}, 0, .., 0) \in \mathcal{R}_q^{k+1}$ and $\overline{ct}_2 = (c_{2,0}, 0, c_{2,1}, 0, .., 0) \in \mathcal{R}_q^{k+1}$, output ciphertext ct_{add}:

$$ct_{add} = ([c_{1,0} + c_{2,0}]_q, c_{1,1}, c_{2,1}, 0, \ldots, 0) \in \mathcal{R}_q^{k+1}$$

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

B. Paillier Cryptosystem

The paillier cryptosystem [12] is a semi-homomorphic encryption algorithm based on the composite residuosity class problem that supports additive homomorphic operation. We mainly use the following algorithms:

• $\text{Paillier}.\text{KeyGen}(1^{\kappa}) \rightarrow \{pk, sk\}$: Given the security parameter κ and randomly select two big primes p and q, where $|p| = |q| = \kappa$. Compute $N = p \cdot q$ and $\lambda = lcm(p-1, q-1)$. Sample $g \leftarrow \mathbb{Z}^*_{N^2}$ satisfying $\gcd(L(g^{\lambda} \bmod N^2), N) = 1$, where $L(x) = (x-1)/N$. Computer $\mu = L(g^{\lambda} \bmod N^2))^{-1} \bmod N$. Output public key pk and secret key sk:

$$pk = (N, g)$$

$$sk = (\mu, \lambda)$$

• $\text{Paillier}.\text{Encrypt}(pk, m) \rightarrow ct$: Given message $m \in \mathbb{Z}_N$, sample $r \leftarrow \mathbb{Z}^*_N$. Output ciphertext ct:

$$ct = E_{pk}(m) = g^m \cdot r^N \bmod N^2$$

• $\text{Paillier}.\text{Decrypt}(sk, ct) \rightarrow m$: Given ciphertext ct and secret key sk, output plaintext m:

$$m = D_{sk}(ct) = L(ct^{\lambda} \bmod N^2) \cdot \mu \bmod N$$

• $\text{Paillier}.\text{Add}(ct_1, ct_2) \rightarrow ct_{add}$: Given two arbitrary ciphertexts $ct_1 = E_{pk}(m_1)$ and $ct_2 = E_{pk}(m_2)$. Output ciphertext ct_{add}:

$$ct_{add} = ct_1 \cdot ct_2 \bmod N^2$$

C. Polynomial Interpolation

Polynomial interpolation [33] is a method of approximating a function $f(x)$ by a polynomial, using the values of $f(x)$ at a number of known points in an interval. It can be expressed as:

• $\text{Poly}.\text{Inter}((x_0, y_0), (x_1, y_1), \ldots, (x_z, y_z)) \rightarrow P(x)$: Given $z+1$ points $\{(x_i, y_i)\}_{i=0}^{z}$, where $y_i = f(x_i)$. Output a polynomial (called an interpolating polynomial) $P(x)$:

$$P(x) = a_k x^k + a_{k-1} x^{k-1} + \cdots + a_0$$

which satisfies $y_i = P(x_i)(i = 0, 1, \ldots z)$.

Theorem 1. Given $z+1$ points $\{(x_i, y_i)\}_{i=0}^{z}$, if x_i are all different, then there exists a unique polynomial $P(x)$ with degree no more than z such that $y_i = P(x_i)(i = 0, 1, \ldots z)$ holds.

There are many polynomial interpolation methods, such as Lagrange interpolation [34] and Newton interpolation [35].

III. System Model and Threat Model

A. System Model

As shown in Fig. 1, the system model consists of a server and n clients $U_{i \in [0, n-1]}$, denoted $U = \{U_0, U_1, \ldots, U_{n-1}\}$.

• Clients: each client U_i generates its own public and secret keys (step ①). U_i encrypts model update m_i using public key before uploading it to server (step ②), and decrypts the aggregated ciphertext CT_{agg} with the secret key after downloading it from the server to get the aggregated result m_{agg} (step ④).

• Server: It initializes the system parameters and aggregates ciphertexts from clients (step ③).

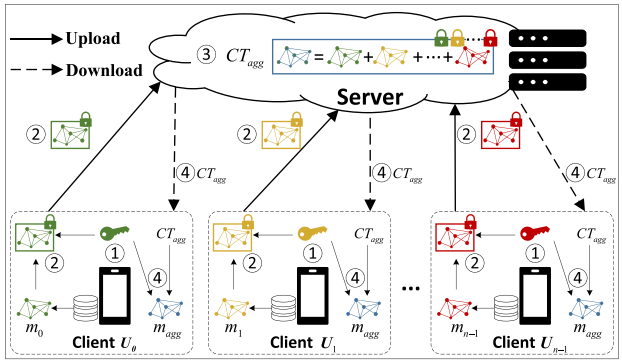


Fig. 1. System model.

B. Threat Model

In SAMK, we consider both clients and the server to be honest but curious. They perform SAMK honestly but attempt to infer and analyze received messages or publicly available ciphertexts, i.e., the server may try to obtain sensitive information about clients by inferring their model updates and aggregation results; clients may try to obtain other clients' sensitive information by inferring their model updates. We assume no collusion exists between clients and the server, but clients may collude to jointly infer sensitive information about non-colluding clients. In addition, all the communication channels between the clients and the server are insecure and any entity (server, clients and outsiders) can intercept the transmitted information.

Under the above threat model, we propose the following privacy requirements:

• Confidentiality of clients' model updates: Any attacker (dishonest insiders and outsiders) might obtain training data information through model updates. To protect data privacy, each client's model updates should be encrypted and cannot be decrypted by other entities.

• Confidentiality of aggregated result: The aggregated result may contain some clients' proprietary information. Hence, the aggregated result should remain encrypted and cannot be decrypted by any attacker (including the server) except for the clients participating in the training.

IV. SAMK Framework

A. Technical Intuition

1) Another Way to Decrypt BFV Ciphertext: In the above system, the challenge is that each client's secret key can decrypt CT_{agg} individually. That is, for the ciphertext CT_{agg}, there are n different secret keys that can all decrypt CT_{agg}. We notice that polynomial interpolation can achieve this function. For a polynomial with degree z, all clients can recover it as long as they have any set of its $z+1$ points.

For HE, the ciphertext of the RLWE-based cryptosystems is exactly polynomial, so we use it to encrypt model updates, taking BFV as an instance. Note that our scheme is also applicable to other RLWE-based HE.

In secure aggregation based on multi-key BFV cryptosystem, each client U_i has its own BFV keys (pk_i, sk_i) and encrypts

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

model update m_i by pk_i. The ciphertext is

$$ct = (c_{i,0}, c_{i,1}) = ([\Delta[m_i]_t + u_i p_{i,0} + e_{i,0}]_q, [u_i p_{i,1} + e_{i,1}]_q)$$

let $f_i(x) = sk_i \cdot c_{i,1} = s_i[u_i p_{i,1} + e_{i,1}]_q$, we can perform the following computation to obtain m_i:

$$m_i = \left[\left\lfloor \frac{t \cdot [c_{i,0} + f_i(x)]_q}{q} \right\rceil\right]_t$$

Similarly, after aggregation, each client can obtain m_{agg} by computing:

$$m_{agg} = \left[\left\lfloor \frac{t \cdot [CT_{agg} + F(x)]_q}{q} \right\rceil\right]_t$$

where $m_{agg} = [\sum_{i=0}^{n-1} m_i]_t$, $CT_{agg} = [\sum_{i=0}^{n-1} c_{i,0}]_q$ and $F(x) = \sum_{i=0}^{n-1} f_i(x)$.

Completing the above decryption process requires the client to know $F(x)$. We let each client learn N different points of $F(x)$, and then they can learn $F(x)$ by polynomial interpolation. Since $F(x) = \sum_{i=0}^{n-1} f_i(x)$, the N points of $F(x)$ can be obtained by aggregating N points of all $f_i(x)$. That is, for N independent variables $X = \{x_0, x_1, \ldots, x_{N-1}\}$, we can compute the N values of $F(x)$ as follows:

$$\begin{aligned} F(x_0) &= f_0(x_0) + f_1(x_0) + \cdots + f_{n-1}(x_0) \\ F(x_1) &= f_0(x_1) + f_1(x_1) + \cdots + f_{n-1}(x_1) \\ &\vdots \\ F(x_{N-1}) &= f_0(x_{N-1}) + f_1(x_{N-1}) + \cdots + f_{n-1}(x_{N-1}) \end{aligned}$$

After obtaining the N values of $F(x)$, $\{(x_0, F(x_0)), (x_1, F(x_1)), \ldots, (x_{N-1}, F(x_{N-1}))\}$ is the N points of $F(x)$, which the client can use to recover $F(x)$ by polynomial interpolation.

Therefore, each client needs to compute N points of $f_i(x)$ and send them to the server for aggregation to obtain N points of $F(x)$. In decryption, the server sends these N points to all clients, so that each client can independently recover $F(x)$ using these N points:

$$\text{Poly.Inter}((x_0, F(x_0)), (x_1, F(x_1)), \ldots, (x_{N-1}, F(x_{N-1}))) \rightarrow F(x)$$

Then, the client can decrypt the CT_{agg} individually using $F(x)$.

2) Resist to Honest-but-Curious and Collusion Attacks: According to our privacy requirements (Section III-B), m_i cannot be obtained by any entity except U_i and m_{agg} can only be obtained by the clients. $f_i(x)$ and $F(x)$ are important information for obtaining m_i and m_{agg}, respectively. We let the client $U_w (w \neq i)$ have access to N points of $F(x)$ and at most $N-1$ points of $f_i(x)$, and other entities (the server and outsiders) have access to at most $N-1$ points of $f_i(x)$ and $F(x)$, so as to satisfy the privacy requirement. Specifically, we encrypted all points of $f_i(x)$ and $F(x)$ using the paillier cryptosystem. For $f_i(x)$, we encrypt one point of it with the client's paillier key and the remaining $N-1$ points encrypted by the server's paillier key. By decrypting, the server can obtain at most $N-1$ points, the client can obtain at most 1 point, and outsiders cannot obtain any point. For $F(x)$, the ciphertexts of its points are the aggregated ciphertexts of the points of $f_i(x)$. By decrypting, the server can obtain at most $N-1$ points, the client can obtain at most 1 point, and outsiders cannot obtain any point. Then the server sends the $N-1$ points to the client, so that the client can know the N points of $F(x)$.

All clients can get different sets of N points by increasing the independent variables. For example, the set of independent variables is $\{x_0, x_1, \ldots, x_{N-1}, x_N\}$, U_1 and U_2 can get N points corresponding to $\{x_0, x_1, \ldots, x_{N-2}, x_{N-1}\}$ and $\{x_0, x_1, \ldots, x_{N-2}, x_N\}$, respectively. In this way, all clients need to compute more values of $f_i(x)$. To reduce the clients' computational burden, it is sufficient to compute only N points, i.e., all clients get the same N points for polynomial interpolation to recover $F(x)$. Furthermore, this strengthens the security against client collusion attacks by minimizing the leaked information, ensuring that all clients $U_w (w \neq i)$ get only a single point identical to $f_i(x)$.

We assume that the $N-1$ points decrypted by the server is $\{(x_t, F(x_t))\}_{t=0}^{N-2}$ and the one point decrypted by client is $(x_{N-1}, F(x_{N-1}))$

3) Limit Interactions and Support Clients Offline: In order to reduce the interaction, we let the client get N points without interacting with other clients. When encrypting $f_i(x)$'s points, U_i encrypts $f_i(x_{N-1})$ by each client's paillier public key separately, generating n ciphertexts, so that after aggregation, all clients can decrypt one of the aggregated paillier ciphertexts to obtain $F(x_{N-1})$. Combined with the $N-1$ points $\{(x_t, F(x_t))\}_{t=0}^{N-2}$ decrypted by the server, each client can eventually obtain N points $\{(x_t, F(x_t))\}_{t=0}^{N-1}$ individually.

It is possible to support clients offline. All clients can go offline anytime, both before and after uploading ciphertext. If the client U_w goes offline after uploading the ciphertext, U_w can still be able to get the aggregated results upon reconnecting. If the client U_w goes offline before uploading the ciphertext, U_w should not get the aggregated result, meaning that we need to make it impossible for U_w to obtain all the N points. To achieve this, the server can either not aggregate the paillier ciphertexts encrypted with U_w's paillier public key so that U_w can get at most $N-1$ points decrypted by the server, or the server aggregates all ciphertexts, and after decrypting $N-1$ points, encrypts them separately using the online clients' key, so that U_w can only get one point by decrypting one of the aggregated paillier ciphertexts. For the latter method, the online client needs to decrypt N points when decrypting, imposing a higher decryption burden, so we prefer the former.

4) Putting it all Together: We summarize our scheme and name it SAMK, a secure aggregation scheme under multiple keys. In Section IV, we describe the construction of SAMK in detail.

B. Construction of SAMK

SAMK consists of five phases: setup, keygen, encrypt, aggregate and decrypt. The server performs setup, keygen and aggregate. Clients perform keygen, encrypt, and decrypt. Each phase is executed by each entity individually. Below is the detailed construction of these phases. Some notations used are

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

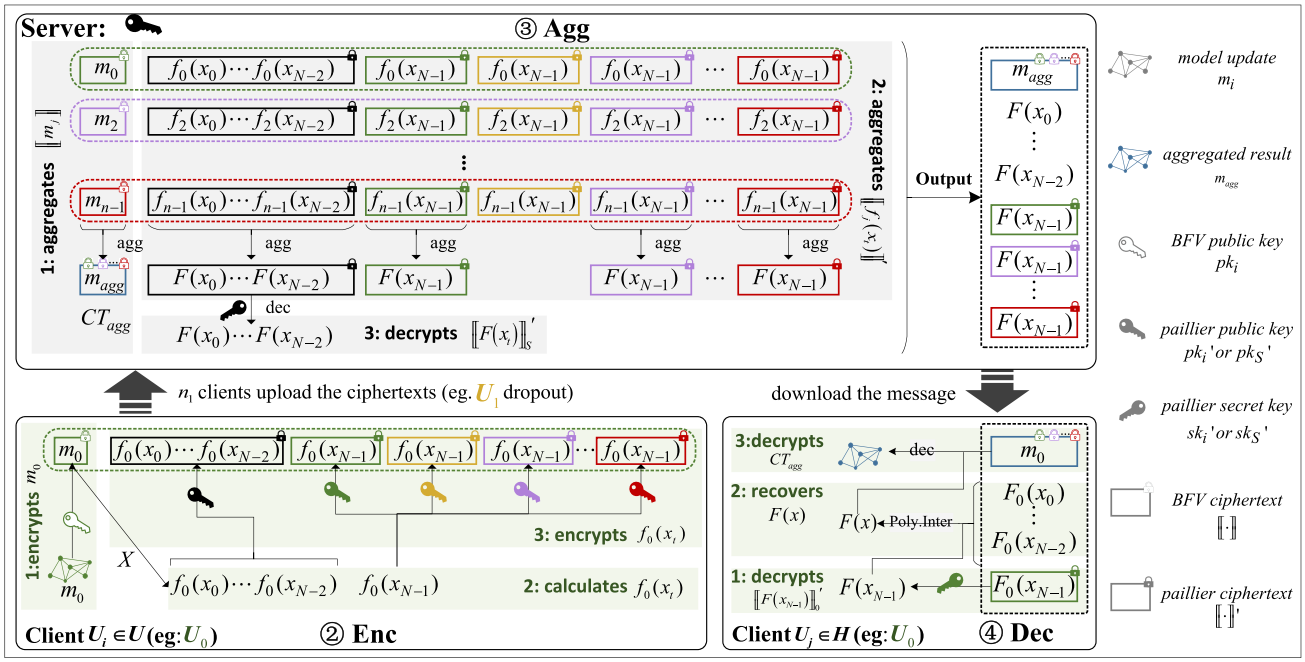


Fig. 2. Process of encryption, aggregation and decryption in SAMK.

TABLE I
NOTATIONS USED IN SAMK

Notaions	Descriptions
U	The set of all clients
H	The set of clients that have uploaded ciphertext
n	Number of clients in set U, $n = \|U\|$
n_1	Number of clients in set H, $n_1 = \|H\|$
m_i	Local model update
X	The set of independent variables
PP	Public parameter
pk_i, sk_i	BFV public and secret key of client U_i
pk_i', sk_i'	Paillier public and secret key of client U_i
pk_S', sk_S'	Paillier public and secret key of the server
$[\![\cdot]\!]$	Ciphertext of model update
$[\![\cdot]\!]'$	Ciphertext of polynomial value
CT_{agg}	Aggregate ciphertext of model updates
m_{agg}	Aggregate result of model updates

represented in Table I. For ease to understand, in Fig. 2, we show the encrypt, aggregate and decrypt processes in detail.

Setup: In setup, the server initializes the security parameters of the BFV and paillier cryptosystem, and generates the set of independent variables X and the public parameters PP. The details are shown in Algorithm 1.

In SAMK, paillier cryptosystem is used to encrypt points. Since the set of independent variables X is public, the paillier cryptosystem actually encrypts the polynomial value $f_i(x_t)$ of the BFV ciphertext. In order to ensure that the $f_i(x_t)$ are all in the paillier's plaintext space, κ should satisfy $\frac{(q-1)\cdot(x_{\max}^N-1)}{x_{\max}-1} < 2^{2\kappa}$, where $x_{\max}$ is the maximum element in X and $\frac{(q-1)\cdot(x_{\max}^N-1)}{x_{\max}-1}$ is the maximum polynomial value of $f_i(x_t)$. However, when $f_i(x_t)$ is particularly large, κ is also large, leading to very low paillier encryption and decryption efficiency. To keep κ small, when $f_i(x_t)$ is large, paillier can encrypt the result of $f_i(x_t)$ mod f', instead of encrypting $f_i(x_t)$, where f' is the maximum value that the paillier plaintext space can hold. Moreover, the quotient of $f_i(x_t)$ divided by f' is disclosed, which is mainly used by the client to recover $F(x_t)$ after obtaining the result of $F(x_t)$ mod f'.

Algorithm 1: Setup.

Public Input: Security parameter λ
Public Output: PP
Server:
1: generates BFV public parameter pp by MK-BFV . Setup(1^λ):

$$pp = \{N, q, t, \varphi, \chi\}$$

2: randomly selects N different $x_t \in \mathbb{Z}$, lets $X = \{x_t\}_{t=0}^{N-1}$;
3: sets the Paillier security parameter κ;
4: lets $PP = \{pp, \kappa, X\}$;
5: broadcasts PP.

KeyGen: In key generation, each entity performs the key generation algorithm individually. The client U_i generates its own BFV key pair (pk_i, sk_i) and paillier key pair (pk_i', sk_i'). The server generates its own paillier key pair (pk_S', sk_S'). The details are shown in Algorithm 2.

After all entities generate their keys, they broadcast their paillier public keys used for encryption. The BFV keys are mainly used to encrypt model updates, and paillier keys are mainly used to encrypt the polynomial value of the BFV ciphertext. The server only aggregates ciphertexts and does not have the model update, so it does not need BFV keys.

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: KeyGen.

Public Input: PP
Public Output: $\{pk_i, {pk_i}'\}_{i=0}^{n-1}, {pk_S}'$
Private Output: $\{sk_i, {sk_i}'\}_{i=0}^{n-1}, {sk_S}'$
Each client U_i**:**
1: generates $\{sk_i, pk_i\} \leftarrow \text{MK-BFV}.\text{KeyGen}(pp)$:

$$sk_i = s_i$$

$$pk_i = (p_{i,0}, p_{i,1}) = ([-(s_i \cdot a_i + e_i)]_q, a_i)$$

2: generates $\{{pk_i}', {sk_i}'\} \leftarrow \text{Paillier}.\text{KeyGen}(1^\kappa)$:

$${pk_i}' = (N_i, g_i)$$

$${sk_i}' = (\mu_i, \lambda_i)$$

3: broadcasts ${pk_i}'$.
Server:
1: generates $\{{pk_S}', {sk_S}'\} \leftarrow \text{Paillier}.\text{KeyGen}(1^\kappa)$:

$${pk_S}' = (N_S, g_S)$$

$${sk_S}' = (\mu_S, \lambda_S)$$

2: broadcasts ${pk_S}'$.

Encrypt: We assume that each client U_i has been trained to obtain local model update m_i before encryption. In encryption, each client U_i encrypts m_i using BFV, and encrypts N polynomial values of BFV ciphertext using paillier. $N-1$ values are encrypted with the server's paillier public key, and the remaining value is encrypted with all clients' paillier public keys separately. We use $[\![\cdot]\!]$ and $[\![\cdot]\!]'$ to denote the ciphertext of model update and polynomial value, respectively.

As shown in Algorithm 3, there are 4 main steps in the encryption phase. 1) The client U_i encrypts its own model update m_i using its own BFV public key pk_i, obtaining the ciphertext $[\![m_i]\!]$. 2) U_i computes the polynomial $f_i(x) = sk_i \cdot c_{i,1}$ using sk_i and c_i. 3) Using the N values in X as independent variables, computes the N different values of $f_i(x)$: $f_i(x_0)$, $f_i(x_1), \ldots, f_i(x_{N-1})$. 4) For $f_i(x_0)$, $f_i(x_1)$,..., $f_i(x_{N-2})$, U_i encrypts them using the server's paillier public key ${pk_S}'$, obtaining the ciphertexts: $[\![f_i(x_0)]\!]_S{}'$, $[\![f_i(x_1)]\!]_S{}'$,..., $[\![f_i(x_{N-2})]\!]_S{}'$. For $f_i(x_N-1)$, U_i encrypts it using each client $U_v \in U$'s paillier public key ${pk_v}'$, obtaining the ciphertexts: $[\![f_i(x_{N-1})]\!]_0{}'$, $[\![f_i(x_{N-1})]\!]_1{}'$,..., $[\![f_i(x_{N-1})]\!]_{n-1}{}'$.

After encryption, the client U_i sends ciphertexts $[\![m_i]\!]$, $\{[\![f_i(x_0)]\!]_S{}', [\![f_i(x_1)]\!]_S{}', \ldots, [\![f_i(x_{N-2})]\!]_S{}'\}$ and $\{[\![f_i(x_{N-1})]\!]_0{}'$, $[\![f_i(x_{N-1})]\!]_1{}', \ldots, [\![f_i(x_{N-1})]\!]_{n-1}{}'\}$ to the server for aggregation. Some clients may be offline and not uploading the ciphertext. We assume that there are n_1 clients who upload the ciphertext, denoting such the set of clients as H, $|H| = n_1$. For convenience, we use j denotes the client's identity in H, that is, $H = \{U_j\}_{j=0}^{n_1-1}$.

In FL, the trained local model generally contains many parameters, meaning the model update is generally a multi-dimensional vector containing all model parameters. Assuming that the trained local model contains l parameters, i.e., $m_i = \{m_{i,0}, m_{i,1}, \ldots m_{i,l-1}\}$, we need to encrypt them all. If we encrypt them directly, we will generate l BFV ciphertexts. For efficiency, batch encoding can be used in BFV encryption. A BFV plaintext polynomial can hold at most N data; encoding N model parameters into a plaintext can encrypt them all at once. Each client's BFV ciphertexts can be reduced to $\lceil l/N \rceil$. Refers to [36] for more details. Additionally, as model parameters are typically non-integer values, they need to be multiplied by a scaling factor ϕ to become integers before encryption using BFV. The final result can be obtained by dividing the decryption result by ϕ.

For a BFV ciphertext, each client U_i encrypts $N-1$ polynomial values $\{f_i(x_t\}_{t=0}^{N-2}$ with the server's key ${pk_S}'$, generating $N-1$ paillier ciphertexts. In paillier, a plaintext space N_S can hold up to k plaintexts, where k satisfies $\sum_{t=0}^{k-1} |f_i(x_t)| \leq |N_S|$. $|N_S|$ and $|f_i(x_t)|$ is the bit length of N_S and plaintext $f_i(x_t)$, respectively. Thus, each client can pack k plaintexts into a single plaintext for efficiency. However, the aggregation result can be affected if multiple plaintexts are packed together directly due to the carrying generated by the aggregation operation (shown in Fig. 3(a)). Therefore, we reserve enough bits in

Algorithm 3: Encrypt.

Public Input: PP, $\{pk_i, {pk_i}'\}_{i=0}^{n}{}^{1}$, ${pk_S}'$
Private Input: $\{m_i\}_{i=0}^{n-1}$
Public Output:

$$\left\{[\![m_j]\!], \{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}\right\}_{j=0}^{n_1-1}$$

Each client U_i**:**
1: generates $c_i \leftarrow \text{BFV}.\text{Encrypt}(pk_i, m_i)$:

$$\begin{aligned} c_i &= (c_{i,0}, c_{i,1}) \\ &= ([\Delta[m_i]_t + u_i p_{i,0} + e_{i,0}]_q, [u_i p_{i,1} + e_{i,1}]_q) \end{aligned}$$

lets $[\![m_i]\!] = c_{i,0}$
2: computes $f_i(x) = sk_i \cdot c_{i,1}$.
3: computes the polynomial values of $f_i(x)$:
foreach $x_t \in X$ **do**
computers $f_i(x_t) \in \mathbb{Z}$;
4: encrypts $f_i(x_t)$:
foreach $t \in [0, 1, \ldots N-1]$ **do**
if $t < N-1$ **then**
generates
$[\![f_i(x_t)]\!]' \leftarrow \text{Paillier}.\text{Encrypt}({pk_S}', f_i(x_t))$:

$$[\![f_i(x_t)]\!]_S{}' = g_S^{f_i(x_t)} \cdot r_i^{N_S} \bmod {N_S}^2$$

else
foreach $v \in [0, 1, \ldots n-1]$ **do**
generates
$[\![f_i(x_t)]\!]_v{}' \leftarrow \text{Paillier}.\text{Encrypt}(pk_v', f_i(x_t))$:

$$[\![f_i(x_t)]\!]_v{}' = g_v^{f_i(x_t)} \cdot r_i^{N_v} \bmod {N_v}^2$$

Online client $U_j \in H$**:**
1: sends $[\![m_j]\!], \{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}$ to the server.

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

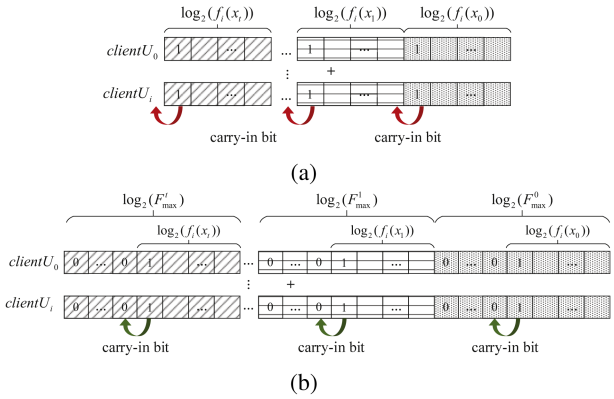


Fig. 3. Aggregate packaged plaintext.

the high bits of each plaintext to accommodate the carrying bits after aggregation, making the aggregation result correct for each plaintext in the packet. As presented in Fig. 3(b), we extend $f_i(x_t)$ to $|F^t_{\max}|$ bits by filling in the high bits with 0, where $F^t_{\max} = \frac{(q-1)\cdot(x_t^N-1)\cdot n}{x_t-1}$. $F^t_{\max}$ is the maximum possible aggregation result for n clients' $f_i(x_t)$, so $|F^t_{\max}|$ bits can ensure that there is enough space to accommodate the aggregation result for $\sum_{i=0}^{n} f_i(x_t)$. Eventually k' plaintexts are packed into a single plaintext, where $\sum_{t=0}^{k'-1} |F^t_{\max}| \le |N_S|$ and $\sum_{t=0}^{k'} |F^t_{\max}| > |N_S|$.

Aggregate: In aggregation, the server performs homomorphic addition operation on model updates' ciphertexts $\{[\![m_j]\!]\}_{j=0}^{n_1-1}$ and polynomial values' ciphertexts $\{\{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}\}_{j=0}^{n_1-1}$. Then it decrypts part of aggregated ciphertexts to obtain $N-1$ polynomial values $\{F(x_t)\}_{t=0}^{N-2}$.

As shown in in Algorithm 4, there are 3 main steps in the aggregation phase. 1) The server aggregates all clients' $[\![m_j]\!]$ to obtain the aggregated ciphertext CT_{agg}. 2) The server aggregates all clients' ciphertexts $[\![f_j(x_0)]\!]_S{}'$, $[\![f_j(x_1)]\!]_S{}', \ldots, [\![f_j(x_{N-2})]\!]_S{}'$ respectively to get the aggregated ciphertexts $[\![F(x_0)]\!]_S{}'$, $[\![F(x_1)]\!]_S{}', \ldots, [\![F(x_{N-2})]\!]_S{}'$:

$$[\![F(x_0)]\!]_S{}' = [\![f_0(x_0)]\!]_S{}' \cdot \ldots \cdot [\![f_{n_1-1}(x_0)]\!]_S{}' \bmod {N_S}^2$$

$$[\![F(x_1)]\!]_S{}' = [\![f_0(x_1)]\!]_S{}' \cdot \ldots \cdot [\![f_{n_1-1}(x_1)]\!]_S{}' \bmod {N_S}^2$$

$$\vdots$$

$$[\![F(x_{N-2})]\!]_S{}' = [\![f_0(x_{N-2})]\!]_S{}' \cdot \ldots \cdot [\![f_{n_1-1}(x_{N-2})]\!]_S{}' \bmod {N_S}^2$$

Then the server aggregates all clients' ciphertexts $[\![f_j(x_{N-1})]\!]_0{}'$, $[\![f_j(x_{N-1})]\!]_1{}', \ldots, [\![f_j(x_{N-1})]\!]_{n_1-1}{}'$ respectively to get the aggregated ciphertexts $[\![F(x_{N-1})]\!]_0{}'$, $[\![F(x_{N-1})]\!]_1{}', \ldots, [\![F(x_{N-1})]\!]_{n_1-1}{}'$:

$$[[F(x_{N-1})]]_0{}' = [[f_0(x_{N-1})]]_0{}' \cdot \ldots \cdot [[f_{n_1-1}(x_{N-1})]]_0{}' \bmod {N_0}^2$$

$$[[F(x_{N-1})]]_1{}' = [[f_0(x_{N-1})]]_1{}' \cdot \ldots \cdot [[f_{n_1-1}(x_{N-1})]]_1{}' \bmod {N_1}^2$$

$$\vdots$$

$$[[F(x_{N-1})]]_{n_1-1}{}' = [[f_0(x_{N-1})]]_{n_1-1}{}' \cdot \ldots \cdot [[f_{n_1-1}(x_{N-1})]]_{n_1-1}{}' \bmod {N_{n_1-1}}^2$$

3) Finally, the server decrypts the ciphertexts $[\![F(x_0)]\!]_S{}'$, $[\![F(x_1)]\!]_S{}', \ldots, [\![F(x_{N-2})]\!]_S{}'$ using its paillier key $sk_S{}'$ to obtain $F(x_0), F(x_1), \ldots, F(x_{N-2})$. If the packing technique is used in the encryption, the plaintext needs to be unpacked after the server decrypts the polynomial values' ciphertexts.

After aggregation, the server sends $\{CT_{agg}, F(x_0), F(x_1), \ldots, F(x_{N-2}), [\![F(x_{N-1})]\!]_0{}'\}$, $\{CT_{agg}, F(x_0), F(x_1), \ldots, F(x_{N-2}), [\![F(x_{N-1})]\!]_1{}'\}$,..., $\{CT_{agg}, F(x_0), F(x_1), \ldots, F(x_{N-2}), [\![F(x_{N-1})]\!]_{n_1-1}{}'\}$ to clients U_0, U_1,..., U_{n_1-1} respectively.

Decrypt: In decryption, each client $U_j \in H$ has received the message $\{CT_{agg}, F(x_0), F(x_1), \ldots, F(x_{N-2}), [\![F(x_{N-1})]\!]_j{}'\}$ from the server. U_j is able to independently decrypt $[\![F(x_{N-1})]\!]_j{}'$, and then combining $F(x_{N-1})$ and $F(x_0), F(x_1), \ldots, F(x_{N-2})$, U_j can independently decrypt CT_{agg} to obtain the aggregated result m_{agg}.

Algorithm 4: Aggregate.

Public Input:
$PP, \left\{[\![m_j]\!], \{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}\right\}_{j=0}^{n_1-1}$

Private Input: $sk_S{}'$

Public Output: CT_{agg}, $\left\{[\![F(x_{N-1})]\!]_j{}'\right\}_{j=0}^{n_1-1}$, $\{F(x_t)\}_{t=0}^{N-2}$

Server:

1: aggregates $\{[\![m_j]\!]\}_{j=0}^{n_1-1}$:

$$CT_{agg} = \left[\sum_{j=0}^{n_1-1} [\![m_j]\!]\right]_q$$

2: aggregates $\left\{\{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}\right\}_{j=0}^{n_1-1}$:

foreach $t \in [0, 1, \ldots N-2]$ **do**

$$[\![F(x_t)]\!]_S{}' = \prod_{j=0}^{n_1-1} [\![f_j(x_t)]\!]_S{}' \bmod {N_S}^2$$

foreach $v \in [0, 1, \ldots n_1-1]$ **do**

$$[\![F(x_{N-1})]\!]_v{}' = \prod_{j=0}^{n_1-1} [\![f_j(x_{N-1})]\!]_v{}' \bmod {N_v}^2$$

3: decrypts $[\![F(x_t)]\!]_S{}'$ $(t = 0, 1, \ldots, N-2)$:

foreach $t \in [0, 1, \ldots N-2]$ **do**

$F(x_t) \leftarrow \text{Paillier}.\text{Decrypt}(sk_S{}', [\![F(x_t)]\!]_S{}')$

4: sends $\left\{CT_{agg}, \{F(x_t)\}_{t=0}^{N-2}, \left\{[\![F(x_{N-1})]\!]_j{}'\right\}_{j=0}^{n_1-1}\right\}$:

foreach $j \in [0, 1, \ldots n_1-1]$ **do**

sends $\left\{CT_{agg}, \{F(x_t)\}_{t=0}^{N-2}, [\![F(x_{N-1})]\!]_j{}'\right\}$ to $U_j \in H$

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: Decrypt.

Public Input: $PP, CT_{agg}, \{[\![F(x_{N-1})]\!]_j{}'\}_{j=0}^{n_1-1}, \{F(x_t)\}_{t=0}^{N-2}$
Private Input: $\{sk_j{}'\}_{j=0}^{n_1-1}$
Private Output: m_{agg}
Each client $U_j \in H$**:**
1: decrypts $[\![F(x_{N-1})]\!]_j{}'$:

$$F(x_{N-1}) \leftarrow \text{Paillier . Decrypt}(sk_j{}', [\![F(x_{N-1})]\!]_j{}')$$

2: recovers polynomial $F(x)$:

$$F(x) \leftarrow \text{Poly.Inter}((x_0, F(x_0)), \ldots, (x_{N-1}, F(x_{N-1})))$$

3: decrypts CT_{agg}:

$$m_{agg} = \left[\left\lfloor \frac{t \cdot [CT_{agg} + F(x)]_q}{q} \right\rceil\right]_t$$

As shown in Algorithm 5, there are 3 steps in the decryption phase. 1) The client U_j decrypts the ciphertext $[\![F(x_{N-1})]\!]_j{}'$ using its paillier key $sk_j{}'$ to obtain $F(x_{N-1})$. 2) After obtaining $F(x_{N-1})$, combined with the received $N-1$ points $\{F(x_t)\}_{t=0}^{N-2}$, the client U_j has a total of N points of $F(x)$. The client uses these N points for polynomial interpolation to obtain $F(x)$. 3) Finally, the client U_j uses $F(x)$ and CT_{agg} to obtain the aggregation result m_{agg}. If batch encoding is used in the BFV encryption, after decrypting the aggregated ciphertext CT_{agg}, the plaintext m_{agg} needs to be decoded to recover all model parameters. Refers to [36] for more details.

The client U_j independently performs the entire decryption process without interaction with other clients, realizing independent decryption. And all clients in H will get the same decryption result after performing the decryption according to the Algorithm 5. This is because after decrypting the ciphertext, all clients $U_j \in H$ obtain N points $\{(x_t, F(x_t)\}_{t=0}^{N-1}$, which can determine a unique polynomial $F(x)$ according to Theorem 1. Based on the same $F(x)$ and CT_{agg}, the clients in H can obtain the same aggregation result m_{agg}. No other entity not in H can obtain the aggregation result due to the inability to decrypt any of $\{[\![F(x_{N-1})]\!]_j{}'\}_{j=0}^{n_1-1}$. They get at most $N-1$ points sent by the server, which is not enough to recover $F(x)$.

V. Security

In this section, we first describe the correctness of decrypting aggregated ciphertexts independently. Then, we analyze the honset but curious security of SAMK. Finally, we analyze the security of SAMK against clients collusion.

A. Correctness of Decryption Independently

Proposition 1: In SAMK, if all entities perform the algorithm honestly, then each client $U_j \in H$ can decrypt the aggregated ciphertext correctly to obtain the aggregated result.

Proof. After the server honestly performs the aggregation computation, each client $U_j \in H$ can receive the message $\{CT_{agg}, F(x_0), F(x_1), \ldots, F(x_{N-2}), [\![F(x_{N-1})]\!]_j{}'\}$.

In the decryption phase, since $[\![F(x_{N-1})]\!]_j{}'$ is encrypted by the client U_j's paillier public key, U_j can decrypt $[\![F(x_{N-1})]\!]_j{}'$ independently using its paillier secret key to obtain $F(x_{N-1})$. Combining $F(x_{N-1})$ with $F(x_0), F(x_1), \ldots, F(x_{N-2})$ received from the server, each client U_j has N different points individually: $(x_0, F(x_0))$, $(x_1, F(x_1))$,..., $(x_{N-2}, F(x_{N-2}))$, $(x_{N-1}, F(x_{N-1}))$. According to Theorem 1, each client U_j can use these N points to recover the unique polynomial $F(x)$ independently by polynomial interpolation. Since $f_j(x) = sk_j \cdot c_{j,1}$, then $F(x)$ is $\sum_{j=0}^{n_1-1} sk_j \cdot c_{j,1}$:

$$\begin{aligned} F(x) &= f_0(x) + f_1(x) + \cdots + f_{n_1-1}(x) \\ &= sk_0 \cdot c_{0,1} + sk_1 \cdot c_{1,1} + \cdots + sk_{n_1-1} \cdot c_{n_1-1,1} \\ &= \sum_{j=0}^{n_1-1} sk_j \cdot c_{j,1} \end{aligned}$$

After obtaining $F(x)$, the client performs the following decryption computation to obtain the aggregation result m_{agg}:

$$\begin{aligned} m_{agg} &= \left[\left\lfloor \frac{t \cdot [CT_{agg} + F(x)]_q}{q} \right\rceil\right]_t \\ &= \left[\left\lfloor \frac{t \cdot \left[\left[\sum_j^{n_1-1} [\![m_j]\!]\right]_q + \sum_j^{n_1-1} sk_j \cdot c_{j,1}\right]_q}{q} \right\rceil\right]_t \\ &= \left[\left\lfloor \frac{t \cdot \left[\left[\sum_j^{n_1-1} c_{j,0}\right]_q + \sum_j^{n_1-1} sk_j \cdot c_{j,1}\right]_q}{q} \right\rceil\right]_t \end{aligned}$$

The final decryption process is the same as that of the original multi-key BFV ciphertext, so the client can correctly decrypt the aggregated ciphertext.

Thus, if all entities perform the algorithm honestly, each client $U_j \in H$ can correctly decrypt the aggregated ciphertext. During the whole decryption process, the client decrypts independently without the participation of other clients, and finally obtains the correct aggregation result.

B. Honest But Curious Security

Here, we argue that SAMK is secure in the honest but curious setting. The server's view does not leak any information about the aggregation result and all clients' model updates. Any set of less than $n_1 - 1$ client's joint view does not leak information about other clients' model updates. A party's view consists of its internal state and all the messages that party receives from other parties. Given a security parameter λ, a server S and a set U of n clients, let $C \subseteq U \bigcup \{S\}$ and $H \subseteq U$, where H is a set of n_1 clients, we use the random variable $\text{REAL}_C^{U,\lambda}(H)$ to represent the jointed view of C in the SAMK.

Proposition 2: (Honest but Curious security, against curious clients and server) There exists a PPT simulator SIM such that for all λ, U, H and C such that $C \subseteq U \bigcup \{S\}$, $|C| < n_1 - 1$, the

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

output of SIM (i.e., $\mathrm{SIM}_C^{U,\lambda}$) is computationally indistinguishable from the output of $\mathrm{REAL}_C^{U,\lambda}$:

$$\mathrm{REAL}_C^{U,\lambda}(H) \equiv \mathrm{SIM}_C^{U,\lambda}(H)$$

where "$\equiv$" represents computationally indistinguishable.

Proof: We prove the above proposition via a hybrid argument. We will define a simulator SIM through a series of (polynomially many) subsequent modifications to the random variable in $\mathrm{REAL}_C^{U,\lambda}$, so that any two subsequent random variables are computationally indistinguishable.

Hyb_0*:* This random variable is distributed exactly as $\mathrm{REAL}_C^{U,\lambda}$, the joint view of the parties C in a real execution of the SAMK.

Hyb_1*:* In this hybrid, we change the behaviour of simulated honest clients $U_i \in U\backslash C$, so that instead of using pk_i and $pk_i{}'$ to encrypt model update and polynomial values, they use the BFV key ppk_i and paillier key $ppk_i{}'$ to encrypt them respectively, which are generated by simulator honestly running Algorithm 2. The keys are randomly sampled, ensuring this hybrid is indistinguishable from the previous one.

Hyb_2*:* In this hybrid, we change the behaviour of the simulated honest client $U_i \in U\backslash C$, each encrypts a randomly selected vector ω_i using BFV, instead of encrypting the model update m_i. The property of the BFV cryptosystem guarantees that this hybrid is indistinguishable from the previous one.

Hyb_3*:* In this hybrid, we change the behaviour of simulated honest client $U_i \in U\backslash C$, so that each client U_i encrypts a randomly selected the N- dimensional vector η using paillier, instead of encrypting $f_i(x_t)$. Although the server can decrypt the above $N-1$ ciphertexts and $U_i \in C$ can decrypt the above 1 ciphertext, they do not collude to determine the polynomial and the polynomial remains random, which consistent with the privious hybrid. Hence, this hybrid is indistinguishable from the previous one.

Hyb_4*:* In this hybrid, for all clients $U_j \in H\backslash C$, we change their input of aggregation executed by the server with $\{[\![\omega_j]\!], \{[\![\eta_t]\!]_S{}'\}_{t=0}^{N-2}, \{[\![\eta_{N-1}]\!]_i{}'\}_{i=0}^{n-1}\}$, instead of $\{[\![m_j]\!], \{[\![f_j(x_t)]\!]_S{}'\}_{t=0}^{N-2}, \{[\![f_j(x_{N-1})]\!]_i{}'\}_{i=0}^{n-1}\}$, where η_t is the element of η. The paillier and BFV cryptosystem property guarantee that this hybrid is indistinguishable from the previous one.

Hyb_5*:* In this hybrid, we simulate the server to compute $CCT_{agg} = [\sum_{j=0}^{n_1-1} [\![\omega_j]\!]]_q$ insead of computing $CT_{agg} = [\sum_{j=0}^{n_1-1} [\![m_j]\!]]_q$. The property of BFV cryptosystem guarantees that this hybrid is indistinguishable from the previous one.

Hyb_6*:* In this hybrid, we simulate the server to compute $[\![FF(x_t)]\!]_S{}' = \prod_{j=0}^{n_1-1} [\![\eta_t]\!]_S{}' \mod {N_S}^2$ and $[\![FF(x_{N-1})]\!]_v{}' = \prod_{j=0}^{n_1-1} [\![\eta_{N-1}]\!]_v{}' \mod {N_v}^2$ insead of computing $[\![F(x_t)]\!]_S{}' = \prod_{j=0}^{n_1-1} [\![f_j(x_t)]\!]_S{}' \mod {N_S}^2$ and $[\![F(x_{N-1})]\!]_v{}' = \prod_{j=0}^{n_1-1} [\![f_j(x_{N-1})]\!]_v{}' \mod {N_v}^2$. Although the server can decrypt the above $N-1$ ciphertexts and $U_i \in C$ can decrypt the above 1 ciphertext, their non-collusion makes the polynomial remains random and combined with the properties of paillier. This hybrid is indistinguishable from the previous one.

Hyb_7*:* In this hybrid, we simulate the server to decrypt $[\![FF(x_t)]\!]_S{}'$, getting $FF(x_t)$, instead of decrypting $[\![F(x_t)]\!]_S{}'$. The non-collusion between server and clients ensures that this hybrid is indistinguishable from the previous one.

Hyb_8*:* In this hybrid, we simulate the server inputting $\{CCT_{agg}, \{[\![FF(x_{N-1})]\!]_j{}'\}_{j=0}^{n_1}, \{FF(x_t)\}_{t=0}^{N-2}\}$ to decryption executed by clients, instead of $\{CT_{agg}, \{[\![F(x_{N-1})]\!]_j{}'\}_{j=0}^{n_1}, \{F(x_t)\}_{t=0}^{N-2}\}$. The paillier and BFV cryptosystem property and the non-collusion for the server and clients guarantee that this hybrid is indistinguishable from the previous one.

Thus, we can define a PPT simulator SIM sampled from the distribution described above. The argument proves that the output of the SIM is computationally indistinguishable from the output of REAL. Moreover, $n_1 - 1$ clients cannot infer other clients' model updates from the received messages. Hence, the SAMK holds the security property that the curious server and clients learn nothing about others' model updates, and the aggregated result is also not leaked to the curious server.

C. Security Against Clients Collusion

Proposition 3: Collusion between $k < n_1 - 1$ clients does not leak any information about other clients' model updates in SAMK.

Proof: From Proposition 2, we see that the joint view of any set of less than $n_1 - 1$ clients can be simulated without information about the values of the remaining clients. Regardless of parameter settings, clients cannot learn about other clients' model updates, but they may infer other clients' updates by subtracting their plaintexts from aggregated results. The resulting value is the sum of two clients' model updates and is incapable of determining model update for the remaining clients.

Therefore, SAMK is security for $k < n_1 - 1$ clients colluding.

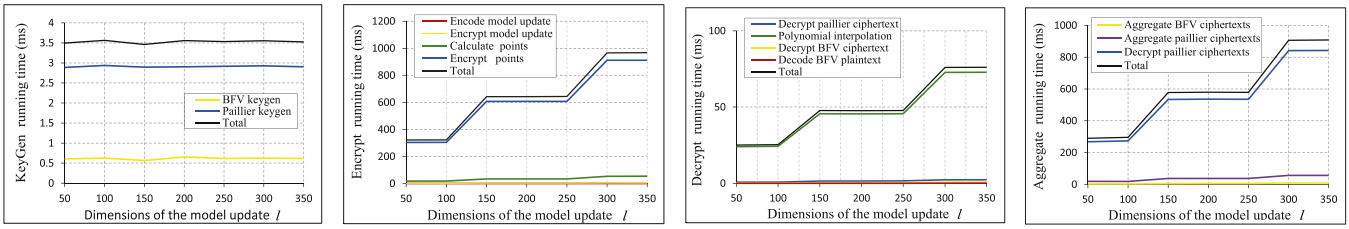
VI. Evalution

In this section, we evaluate the performance of SAMK. We first provide implementation details, and then analyze the performance of SAMK on both client and server. Finally we present the results of SAMK compared with existing schemes.

A. Implementation Details

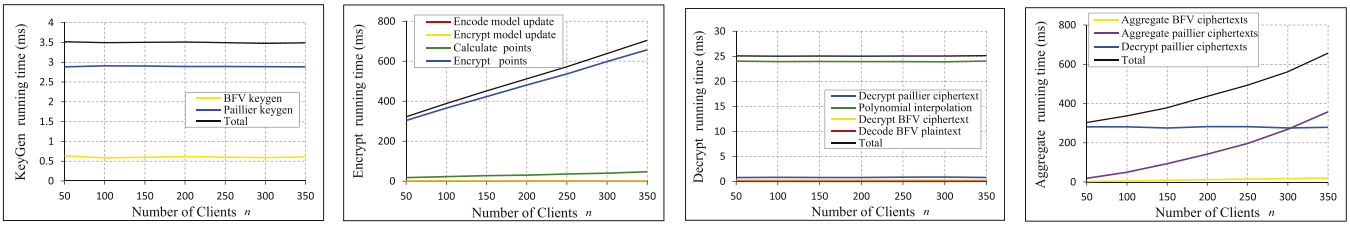
We have implemented SAMK in C++, using GMP [37] for large number operations and Seal [38] for implementing BFV. For polynomial interpolation, we used Newton interpolation. In addition, we implemented 5 existing aggregation schemes for comparison with SAMK: work based on double-masking [4], denoted as DM; work based on single-key HE [18], denoted as SK-HE; work based on sharing public key [20], denoted as SPK-HE; work based on two non-colluding servers [26], denoted as TS-HE; work based on MKHE [30], denoted as MK-HE. Note that we restrict our comparison to secure aggregation mechanism and do not focus on other research issues in these literatures, such as poisoning attacks, data heterogeneity. For [4], we employed Crypto++ [39] to implement DH key agreement and AES-GCM-128 authenticated encryption, and set

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.



(a) KeyGen for client (b) Encryption for client (c) Decryption for client (d) Aggregation for server

Fig. 4. Running time per client and the server ($n = 50$, $N = 128$).



(a) KeyGen for client (b) Encryption for client (c) Decryption for client (d) Aggregation for server

Fig. 5. Running time per client and the server ($l = 50$, $N = 128$).

the threshold to 100. For BFV, according to the Homomorphic Encryption Standardization [40], we set BFV with $N = 1024$ and $\log q = 29$ and use the batch encoding technique in all schemes involving it.

All schemes are performed on a laptop of Intel(R) Core(TM) i7-1165G7 CPU @ 2.80 GHz with 16 GB RAM. The code is implemented by Visual Studio 2022 and MindSpore 1.9 [41]. Since our work is focused on the aggregation approach, the client and server are simulated on a single machine to omit the impact of network communication.

B. Performance Analysis of SAMK

All calculations below assume a server and n clients without offline, each holding a l-dimensional model update. The polynomial degree of BFV is N.

1) Performance Analysis of Client:

Storage cost: $O(l)$: The storage of each client can be divided into 2 parts: 1) its own key and 2) the model update, which has size $O(l)$.

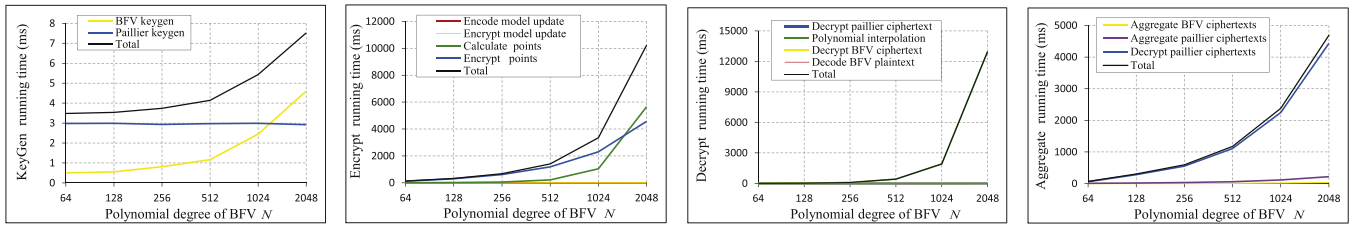
Communication cost: $O(l(N+n))$: The communication of each client can be divided into 2 parts: 1) sending $l(N+n)$ ciphertexts and 2) receiving $2l$ ciphertexts and $l(N-1)$ polynomial values. If packing and batch encoding techniques are used, the communication overhead can reduce. This is related to the size of the paillier's plaintext space, if each paillier plaintext can hold k' points, $\lceil l/N \rceil \cdot (1+n) + \left\lceil \frac{\lceil l/N \rceil \cdot (N-1)}{k'} \right\rceil$ ciphertexts are sent; $2\lceil l/N \rceil$ ciphertexts and $\lceil l/N \rceil \cdot (N-1)$ points are received.

Computation cost: The computation of each client can be divided into 3 phases: key generation, encryption and decryption. In the key generation phase, the BFV key and paillier key are generated. In the encryption phase, it is divided into 4 parts: 1) encoding model update, 2) encrypting model update by BFV, 3) calculating N points for model update's ciphertext and 4) encrypting points by paillier. In the decryption phase, it is also divided into 4 parts: 1) decrypting paillier ciphertexts, 2) polynomial interpolation to recover polynomial of degree $N-1$, 3) decrypting the BFV ciphertext and 4) decoding BFV plaintext. Many complex calculations are involved in these computations. In summary, the total computational cost of the client is mainly related to the dimensions of model update l, the polynomial degree of BFV N, and the number of clients n. Through practical simulations, we analyzed in depth the impact of these three elements on each phase, as shown in Figs. 4, 5 and 6. Note that the parameters of paillier cryptosystem also impact the computation cost. As the paillier plaintext space increases, the number of model parameters a plaintext can hold increases, but the speed of the paillier encryption and decryption decreases. This is mainly concerns the performance of paillier itself, which is not our focus, so we do not analyze it in depth. In our simulation, the paillier key is set to 1024 b.

In key generation, each client generates its key individually, independent of the number of clients and the dimensions of model update, as reflected in Figs. 4(a) and 5(a). The BFV key, a polynomial with degree $N-1$, is affected by the size of N. Fig. 6(a) also shows that when $N > 1024$, the BFV key generation time is mostly more than the 1024-bit paillier key generation time.

In encryption, the computation cost is mainly in calculating points and encrypting points, as seen in Figs. 4(b) and 5(b), their running time increase linearly as n but remain constant with l when $l \leq N$. We have used encoding technique in BFV that k ciphertexts can encrypt kN model parameters. When $l = kN + 1 (k = 1, 2, \ldots)$, the encryption time increases due to generating more ciphertexts to encrypt all the model

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.



(a) KeyGen for client (b) Encryption for client (c) Decryption for client (d) Aggregation for server

Fig. 6. Running time per client and the server ($l = 50$, $n = 50$).

parameters; after that, it remains constant until $l > (k+1)N$. They also increase with N. In Fig. 6(b), their running time rise rapidly with N due to the linear increase in the number of points and the exponential increase in the value of point as N increases. Encoding and encrypting model update take very little time. They are mainly influenced by l and N, which together determine the number of encoded plaintexts and ciphertexts. This influence is almost negligible due to the slight time variation.

In decryption, the computation cost is mainly in polynomial interpolation, recovering a polynomial with $N-1$ degrees, so it increases as N (Fig. 6(c)). This suggests that using an optimized polynomial interpolation implementation would significantly reduce our prototype's runtime. Similar to the encryption, the decryption time remains constant when $kN < l \leq (k+1)N$ $(k = 0, 1, \ldots)$, due to the number of ciphertexts to be decrypted remains the same, as shown in Fig. 4(c). In Fig. 5(c), the decryption time is constant with n. This is because the number of clients mainly affects the number of paillier ciphertexts in encryption, while in decryption, each client decrypts a single paillier ciphertext individually, independent of n.

2) Performance Analysis of Server:

Storage cost: $O(1)$*:* The server only stores its paillier key.

Communication cost: $O(l \cdot n_1 \cdot (N+n))$*:* The communication of each client can be divided into 2 parts: 1) receiving $l(n+N) \cdot n_1$ ciphertexts and 2) sending $2l \cdot n_1$ ciphertexts and $l(N-1) \cdot n_1$ points to n_1 clients. If packing and batch encoding techniques are used and each paillier plaintext can hold k' points, $(\lceil l/N \rceil \cdot (1+n) + \left\lceil \frac{\lceil l/N \rceil \cdot (N-1)}{k'} \right\rceil) \cdot n_1$ ciphertexts are received; $2\lceil l/N \rceil \cdot n_1$ ciphertexts and $\lceil l/N \rceil \cdot (N-1) \cdot n_1$ points are sent.

Computation cost: The computation of the server can be divided into 2 phases: key generation and aggregation. In the key generation phase, the paillier key is generated. In the aggregation phase, it is divided into 3 parts: 1) aggregating BFV ciphertexts, 2) aggregating paillier ciphertexts and 3) decrypting paillier ciphertexts. The total computation cost of the server is also mainly related to the dimensions of model update l, the polynomial degree of BFV N, and the number of clients n. The server's key generation is the same as the client's paillier key generation, so we do not show its simulation results.

In aggregation, the computation cost is mainly in aggregating paillier ciphertexts and decrypting paillier ciphertexts. As seen in Fig. 4(d), the aggregation time also remains constant when $kN < l \leq (k+1)N$ $(k = 0, 1, \ldots)$ due to the same number of ciphertexts being aggregated. Fig. 5(d) shows that the running time of aggregating paillier and BFV ciphertexts increases with n, while the time for decrypting paillier ciphertexts remains constant. This is because the server always decrypts $N-1$ aggregated ciphertext regardless of n, which is strongly influenced by N, as reflected in Fig. 6(d). Both the time for aggregating paillier and BFV ciphertexts also increase with N, but this increase is slight compared to the change in time for decrypting the paillier ciphertexts.

3) Summary: To summarize, in our simulation, both for the client and the server, the performance mainly depends on the number of paillier ciphertexts, which is determined by l, n and N together, where the influence of N and l is more obvious. As N increases, the number of paillier ciphertexts increases, and as N decreases, when l is significantly larger than N, the number of paillier ciphertexts also increases significantly due to the increase in the number of BFV ciphertexts. Therefore, N should be selected appropriately according to l while ensuring security.

C. Performance Analysis by Comparing With Other Schemes

We compare SAMK with other schemes in the following 5 parts: 1) function comparison, 2) performance of the entities, 3) performance of the pre-processing process, 4) performance of the aggregation process and 5) performance of handling client dropouts.

1) Function Comparison: Table II compares the functions of SAMK with existing schemes. It shows that our scheme has richer functions and fewer communication rounds than others. Note that the number of communication rounds refers to the number of interactions between each client and server, between clients, and between servers (if there is more than one server) in a round of aggregation. In HE-based schemes, offline support means that the client can be offline during the encryption, aggregation, and decryption phases. Scheme can resist attacks from honest-but-curious server, meaning that the server does not get model updates and the aggregated result.

2) Performance of the Entities: We compare the performance of the server and the client in terms of storage overhead, communication overhead and computation time.

For the storage overhead, as shown in Table III, all HE-based secure aggregation schemes have less storage burden than DM because that entities in DM need to store information for multi-party computation. Table IV demonstrates communication overhead comparison. SAMK necessitates higher communication

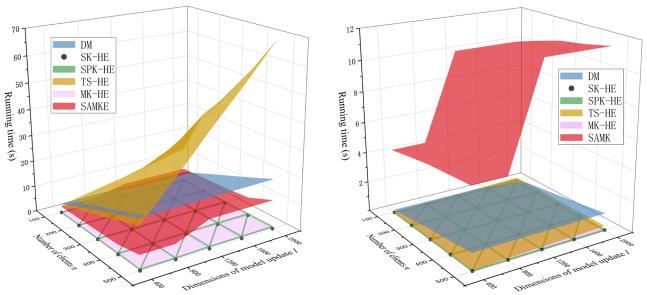
Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FUNCTION COMPARISION

Schemes	DM	SK-HE	SPK-HE	TS-HE	MK-HE	SAMK
Server	Single	Single	Single	Dual	Single	Single
Multi-key	✓	×	✓	✓	✓	✓
HE	×	✓	✓	✓	✓	✓
Honest-but-curious Server	×	✓	×	✓	✓	✓
Honest-but-curious Client	✓	×	✓	✓	✓	✓
Support offline	✓	✓	×	✓	×	✓
Communication round	4	1	2	2	2	1

TABLE III
COMPARISON OF STORAGE OVERHEAD

Schemes	DM	SK-HE	SPK-HE	TS-HE	MK-HE	SAMK
Client	$O(n+l)$	$O(l)$	$O(l)$	$O(l)$	$O(l)$	$O(l)$
Server	$O(n^2+l)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$



(a) Running time of the server (b) Running time of the client

Fig. 7. Running time of entities. Note that the running times of SK-HE, SPK-HE and MK-HE differ by no more than 1 s, and in order to distinguish between them, SK-HE is effectively shown by a scatterplot.

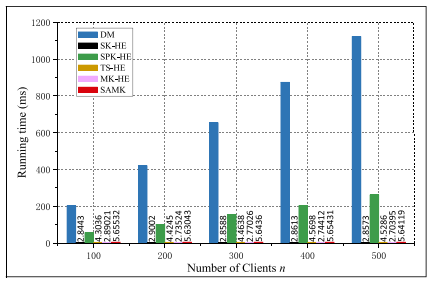


Fig. 8. Running time for pre-processing.

overhead for both the server and the client. The reason is that SAMK needs to send or receive additional polynomial value information compared to other schemes.

We compute the time spent by the server and the client during an aggregation, as demonstrated in Fig. 7. For the server, it spends less time in SAMK as compared to TS-HE, and in most cases less than DM, except when the number of clients is less and the model update dimension is larger; compared to the remaining schemes, it spends more time in SAMK due to aggregate polynomial values' ciphertexts. For the client, it takes the most time in SAMK since it must encrypt both polynomial values and model updates. However, the 3–10 s difference in comparison to other schemes is acceptable in reality.

3) Performance of the Pre-Processing Process: The pre-processing process mainly refers to some operations that can be done in advance before aggregation. In HE-based secure aggregation, this involves key generation, whereas in DM, it includes key generation, key agreement, pseudorandom number generation, and secret sharing, etc.

Fig. 8 illustrates that schemes based on HE exhibit high performance in the pre-processing process. Most schemes only require clients to generate their own keys, independent of the number of clients. However, SPK-HE requires additional steps: after all clients generate their keys, they must collaboratively compute a shared public key by aggregating their individual keys—a process that scales with client count. DM employs MPC and necessitates interactive operations (e.g., key agreement and secret sharing) among all clients, making its performance highly sensitive to the number of clients.

4) Performance of the Aggregation Process: The aggregation process begins when the client processes the model update and ends when the client obtains the plaintext of the aggregated model update.

We evaluate the aggregation time for all schemes under varying numbers of clients and model update dimensions, see Fig. 9(a) and (b). The results indicate that SAMK has advantages over TS-HE in most cases, especially as l and n increase. For other HE-based schemes, SAMK takes more time than them due to its additional paillier encryption for points, whereas they only encrypt the plaintext. This suggests that using an optimized paillier implementation would take less aggregation time than our prototype, or utilizing more efficient additive HE algorithms for point encryption.

Compared with DM, SAMK is better when l is small and conversely, SAMK spends about 15 seconds more than DM in our simulations. As n increases, SAMK always takes about 10 seconds more than DM. In practice, DM requires waiting time for a sufficient number of clients to collaborate on the aggregation results, whereas SAMK allows each client to obtain the aggregation results independently without waiting time.

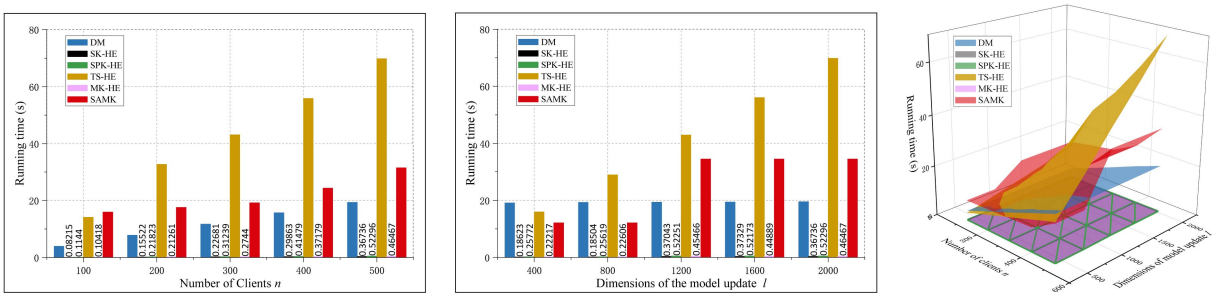
In Fig. 9(c), we show the combined effect of the number of clients and the dimensions of model update on the aggregation runtime for all schemes. SAMK performs better than TS-HE in most cases except when both n and l are small, in which case SAMK also takes more time than DM, but this gap decreases quickly as n increases. This means that SAMK is more adapted to the case with a large number of clients.

5) Performance of Handling Client Dropouts: We compute the aggregation time of schemes that support client dropouts across various dropout rates, as presented in Fig. 10(a). In Fig. 10(a), we assume that the client drops out before uploading

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

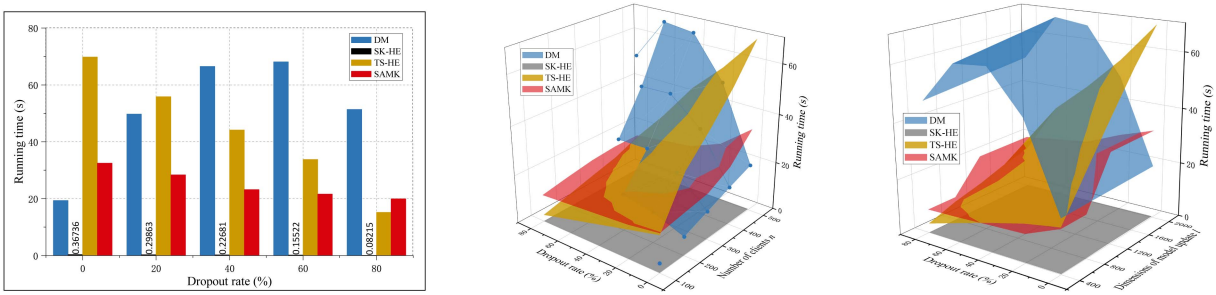
TABLE IV
COMPARISON OF COMMUNICATION OVERHEAD

Schemes	DM	SK-HE	SPK-HE	TS-HE	MK-HE	SAMK
Client	$O(n+l)$	$O(l)$	$O(l+n)$	$O(l)$	$O(nl)$	$O(l(N+n))$
Server	$O(n^2+nl)$	$O(nl)$	$O(nl)$	$O(nl)$	$O(nl)$	$O(l{\cdot}n_1{\cdot}(N{+}n))$



(a) ($l = 2000$) (b) ($n = 500$) (c)

Fig. 9. Running time for aggregating.



(a) ($n = 500$, $l = 2000$) (b) ($l = 2000$) (c) ($n = 500$)

Fig. 10. Running time for aggregating with client dropouts.

the ciphertexts. Except for DM, all schemes' aggregation time decreases with increasing dropout rate. This is because HE-based schemes only aggregate uploaded ciphertexts, meaning that their aggregation time will decrease with fewer ciphertext uploads as the dropout rate rises. However, for DM, dropped clients' masks must be retrieved to remove online clients' paired masks, causing the aggregation time required by DM to increase with the number of dropouts increases. Once the dropout rate exceeds a certain threshold, the aggregation time does not increase but instead decreases. This is because fewer online clients yield less mask removal and masked input vectors, thus reducing the time consumption for aggregation.

In Fig. 10(b) and (c), we show the combined impact of the number of clients and the dropout rate, as well as the dimensions of model update and the dropout rate, on the aggregation runtime for all schemes, respectively. Similarly, assuming the client drops out before uploading the ciphertext. It can be observed that SAMK performs better than TS-HE in most cases except when the drop rate is particularly large and the number of clients is small, i.e., the number of online clients is small, which is also reflected in Fig. 9(c). In addition, SAMK performs better than DM in handling dropouts when the number of clients is large, regardless of the model update's dimensions, and DM does not work properly when the dropout rate makes the number of online clients less than 100 (as shown in Fig. 10(b)), because the mask of online clients cannot be removed.

When clients drop out after uploading the ciphertext, SAMK is unaffected by the dropout rate. It only aggregates the received ciphertexts, regardless of whether the client is online afterwards, as is the case for SK-HE and TS-HE. When clients drop out after uploading the masked input vectors but before uploading the shares, DM is not affected by the dropout rate when the number of online clients does not fall below the threshold; otherwise, it will not perform correctly.

6) Summary: To summarize, SAMK has less pre-processing time and better performance in handling client dropouts when the number of clients is large. It takes longer time in aggregation, mainly than SK-HE, SPK-HE and MK-HE, but it provides higher security against honest-but-curious client-side attacks (unlike SK-HE), and fewer communication rounds in an aggregation round (unlike SPK-HE and MK-HE).

VII. CONCLUSION

In this paper, we focused on the problem of secure aggregation for local model updates in FL, proposing SAMK, a secure

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.

aggregation scheme based on the BFV and paillier cryptosystem. SAMK allows clients to encrypt and decrypt using their own keys and supports them offline anytime. The interaction between entities is also greatly reduced, with the total number of communication rounds in SAMK being only 1 during an aggregation. Moreover, we show SAMK is secure in the semi-honest model and can resist clients collusion attacks. The performance analysis shows that SAMK is comparable in terms of function, pre-processing time and aggregation time (with or without client dropouts), especially in the case of a large number of clients.

In future, we intend to extend it to a malicious adversarial model and consider resistance to server-client collusion attacks.

REFERENCES

[1] L. Bai, H. Hu, Q. Ye, H. Li, L. Wang, and J. Xu, "Membership inference attacks and defenses in federated learning: A survey," *ACM Comput. Surv.*, vol. 57, no. 4, Dec. 2024, Art. no. 89, doi: 10.1145/3704633.

[2] J. Chen, H. Yan, Z. Liu, M. Zhang, H. Xiong, and S. Yu, "When federated learning meets privacy-preserving computation," *ACM Comput. Surv.*, vol. 56, no. 12, Oct. 2024, Art. no. 319, doi: 10.1145/3679013.

[3] Q. Xie et al., "Efficiency optimization techniques in privacy-preserving federated learning with homomorphic encryption: A brief survey," *IEEE Internet Things J.*, vol. 11, no. 14, pp. 24569–24580, Jul. 2024.

[4] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2017, pp. 1175–1191.

[5] S. Zhang, J. Huang, and P. Li, "Analyze and improve differentially private federated learning: A model robustness perspective," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 807–821, 2025.

[6] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.

[7] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2022.

[8] H. Fazli Khojir, D. Alhadidi, S. Rouhani, and N. Mohammed, "FedShare: Secure aggregation based on additive secret sharing in federated learning," in *Proc. 27th Int. Database Engineered Appl. Symp.*, New York, NY, USA, 2023, pp. 25–33, doi: 10.1145/3589462.3589504.

[9] Z. Zhang, J. Li, S. Yu, and C. Makaya, "SAFELearning: Secure aggregation in federated learning with backdoor detectability," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3289–3304, 2023.

[10] J. Tang, H. Xu, M. Wang, T. Tang, C. Peng, and H. Liao, "A flexible and scalable malicious secure aggregation protocol for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4174–4187, 2024.

[11] M. Mansouri, M. Önen, W. B. Jaballah, and M. Conti, "SoK: Secure aggregation based on cryptographic schemes for federated learning," *Proc. Privacy Enhancing Technol.*, vol. 2023, no. 1, pp. 140–157, 2023.

[12] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Berlin, Heidelberg:Springer, 1999, pp. 223–238.

[13] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," 2012. [Online]. Available: https://eprint.iacr.org/2012/144

[14] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Cham: Springer International Publishing, 2017, pp. 409–437.

[15] X. Zhang, A. Fu, H. Wang, C. Zhou, and Z. Chen, "A privacy-preserving and verifiable federated learning scheme," in *Proc. 2020 IEEE Int. Conf. Commun.*, 2020, pp. 1–6.

[16] C. Zhou, A. Fu, S. Yu, W. Yang, H. Wang, and Y. Zhang, "Privacy-preserving federated learning in fog computing," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 10782–10793, Nov. 2020.

[17] Y. Miao et al., "RFed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5814–5827, 2024.

[18] N. Yan et al., "Efficient and straggler-resistant homomorphic encryption for heterogeneous federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2024, pp. 791–800.

[19] F. Wang, H. Zhu, R. Lu, Y. Zheng, and H. Li, "A privacy-preserving and non-interactive federated learning scheme for regression training with gradient descent," *Inf. Sci.*, vol. 552, pp. 183–200, 2021.

[20] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *Int. J. Intell. Syst.*, vol. 37, no. 9, pp. 5880–5901, 2022.

[21] Q. Zhang, S. Jing, C. Zhao, B. Zhang, and Z. Chen, "Efficient federated learning framework based on multi-key homomorphic encryption," in *Proc. Adv. P2P Parallel Grid Cloud Internet Comput.*, Cham: Springer International Publishing, 2022, pp. 88–105.

[22] Z. L. Jiang, H. Guo, Y. Pan, Y. Liu, X. Wang, and J. Zhang, "Secure neural network in federated learning with model aggregation under multiple keys," in *Proc. 8th IEEE Int. Conf. Cyber Secur. Cloud Comput./ 7th IEEE Int. Conf. Edge Comput. Scalable Cloud*, 2021, pp. 47–52.

[23] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.

[24] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.

[25] H. Ku, W. Susilo, Y. Zhang, W. Liu, and M. Zhang, "Privacy-preserving federated learning in medical diagnosis with homomorphic re-encryption," *Comput. Standards Interfaces*, vol. 80, 2022, Art. no. 103583.

[26] Y. Miao et al., "Privacy-preserving asynchronous federated learning under non-IID settings," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5828–5841, 2024.

[27] Z. Ma, J. Ma, Y. Miao, X. Liu, K.-K. R. Choo, and R. H. Deng, "Pocket diagnosis: Secure federated learning against poisoning attack in the cloud," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3429–3442, Nov./Dec. 2022.

[28] M. Zhang, S. Chen, J. Shen, and W. Susilo, "PrivacyEAFL: Privacy-enhanced aggregation for federated learning in mobile crowdsensing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5804–5816, 2023.

[29] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "SecFed: A secure and efficient federated learning based on multi-key homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3817–3833, Jul./Aug. 2024.

[30] X. Hao, C. Lin, W. Dong, X. Huang, and H. Xiong, "Robust and secure federated learning against hybrid attacks: A generic architecture," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1576–1588, 2024.

[31] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Berlin, Heidelberg:Springer, 2003, pp. 37–54.

[32] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2019, pp. 395–412, doi: 10.1145/3319535.3363207.

[33] M. Gasca and T. Sauer, "Polynomial interpolation in several variables," *Adv. Comput. Math.*, vol. 12, no. 4, pp. 377–410, 2000.

[34] T. Sauer and Y. Xu, "On multivariate lagrange interpolation," *Math. Comput.*, vol. 64, no. 211, pp. 1147–1170, 1995.

[35] L. Reichel, "Newton interpolation at Leja points," *BIT*, vol. 30, no. 2, pp. 332–346, 1990.

[36] Z. Brakerski, C. Gentry, and S. Halevi, "Packed ciphertexts in LWE-based homomorphic encryption," in *Proc. Int. Workshop Public Key Cryptography*, Berlin, Heidelberg:Springer, 2013, pp. 1–13.

[37] T. Granlund, "GNU MP: The gnu multiple precision arithmetic library," 2004. [Online]. Available: http://gmplib.org/

[38] SEAL Microsoft (release 4.0), Mar. 2022. [Online]. Available: https://github.com/Microsoft/SEAL

[39] W. Dai, "Crypto++ library," 2023. [Online]. Available: https://www.cryptopp.com

[40] M. Albrecht et al., "Homomorphic encryption standard," HomomorphicEncryption.org, Toronto, Canada, Tech. Rep., Nov. 2018.

[41] Huawei, "Mindspore," 2020. [Online]. Available: https://www.mindspore.cn/

Authorized licensed use limited to: Anhui University. Downloaded on November 02,2026 at 07:31:53 UTC from IEEE Xplore. Restrictions apply.